uno: A Fast, Reproducible, Dependency-Light Rust Engine for MS1-Only Protein Identification

Patrick Garrett
pgarrett@scripps.edu
Department of Chemical Physiology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla, CA 92037, USA

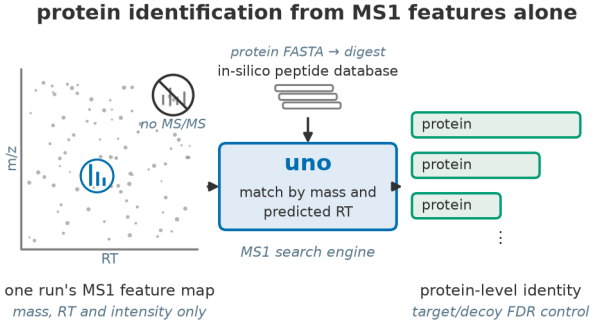
John R. Yates III
jyates@scripps.edu
Department of Chemical Physiology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla, CA 92037, USA

July 2026

Abstract

MS1-only protein identification (DirectMS1) infers peptide and protein identities from precursor mass and retention time (RT) alone, without MS2 fragmentation, enabling short gradients and high throughput. The reference implementation, ms1searchpy, is written in Python with a deep-learning RT predictor. We present uno, an open-source MS1-only search and identification engine in Rust. uno reads the same single-run feature files, so both engines see identical input. On six datasets from the DirectMS1 papers, uno identified more proteins at 1% entrapment false-discovery rate (FDR) than the median of three ms1searchpy replicates, by 3.0 to 39.7%. It ran 5.3 to 7.0 times faster in half the peak memory. On three independent 10 ng HeLa Orbitrap Astral runs, uno returned 1006 to 1448 proteins at 1% entrapment FDR, against ms1searchpy medians of 45 to 47, while running approximately 10 times faster. uno is byte-deterministic. ms1searchpy's depth on identical input varies by up to 40.5% of its median. Two-proteome entrapment finds uno's reported 1% protein FDR conservative on four of six datasets and about twice optimistic on one. On the DirectMS1Quant UPS1 dilution series, uno-derived abundances tracked the spike design (median Spearman $\rho = +0.71$); trace-level differential testing remains outside the validated scope.

Keywords MS1-only proteomics · DirectMS1 · retention time prediction · reproducibility · Rust · open-source software · label-free quantification



1 Introduction

Bottom-up proteomics conventionally identifies peptides from MS2 fragment spectra. Identification from precursor mass and retention time alone is an older idea, developed as the accurate-mass-and-time tag strategy[1], and revived for short-gradient work as DirectMS1[2]. It matches the accurate precursor mass of an LC–MS feature to a theoretical peptide and confirms the assignment with predicted retention time, omitting MS2 entirely. Because no time is spent acquiring fragment spectra, MS1-only workflows support very short gradients (minutes) and correspondingly high throughput, at the cost of an information-poor identification problem. Mass and RT alone are weak discriminators, and identity must be concentrated at the protein level across multiple consistent peptides. The same group had earlier shown that combining enzymes of complementary specificity recovers some of the lost information[3]. Recent work continues to develop MS1-driven identification, including deep-learning frameworks for propagating peptide identity across runs[4].

The established open-source tool for this approach, ms1searchpy[5], couples a precursor mass search to RT prediction by DeepLC[6] and a LightGBM[7] rescoring stage, with protein-level inference by a binomial model. It is effective but has two practical costs that matter at scale. First, its identification counts are not reproducible run to run. Its decoy shuffle and its hyperparameter search both draw on an unseeded random number generator, so repeated searches of the same feature file yield materially different protein lists (Section 3.2). Second, it carries a heavyweight Python and deep-learning dependency stack whose RT-prediction step dominates wall-clock time for a single run, as our own timings show (Section 3.1).

We present uno, an MS1-only protein-identification engine written in Rust. uno's identification path consumes the same single-run feature files that ms1searchpy consumes, so the two engines can be benchmarked on identical input. As a secondary capability, it can read the cross-run aligned intensity matrix produced by the koth feature finder[8], maps identifications onto it, and emits protein-group abundance matrices. uno's design goals are determinism, speed, and minimal external dependencies, while meeting or exceeding the identification depth of ms1searchpy.

This work makes three contributions. First, we provide a clean, open-source Rust reimplementation of DirectMS1-style MS1 search whose output is byte-identical across runs by construction and whose RT prediction runs in-process in pure Rust, so the Python and deep-learning dependency stack is gone (Section 2). Second, we report a same-input head-to-head against ms1searchpy on six datasets taken from the DirectMS1 papers themselves. uno is deeper at 1% entrapment FDR on all six while running several times faster in less memory (Section 3.1), and we quantify how far the reference engine's own answer moves between repeated runs of the same search (Section 3.2). Third, we validate the reported protein FDR of both engines against a two-proteome entrapment probe. The same probe rejected a tested learned protein rescorer whose apparent counts increased, supporting uno's transparent binomial protein score (Sections 3.3 and 3.4).

2 Experimental Section

2.1 Input and pipeline position

uno's identification path consumes a single-run feature file, the same `*.features.tsv` that ms1searchpy takes. Four columns are required, a neutral mass (`massCalib`), `mz`, `charge` and apex retention time (`rtApex`). Everything else is used when present. That covers ion mobility and the shape columns `nIsotopes`, `nScans` and `rt_width`. It also covers the per-feature quality columns emitted by the feature finder: `cosine_score`, `isotope_score`, `combined_score`, `ppm_error`, `neutron_offset`, `isoerror` and `isoerror2`. ms1searchpy additionally requires `nIsotopes` and `nScans`, which uno does not. Single-run input is deliberate. Cross-run alignment collapses many per-run observations into one consensus row and drops those per-feature quality columns, so an aligned matrix cannot be used for identification without changing the information available to the search. The koth-aligned intensity matrix is used only by uno's quantitation path (`uno --matrix`), described below. Because the identification path works one run at a time, RT is in that run's own gradient space and is calibrated per file (Figure 1).

2.2 Peptide database and decoys

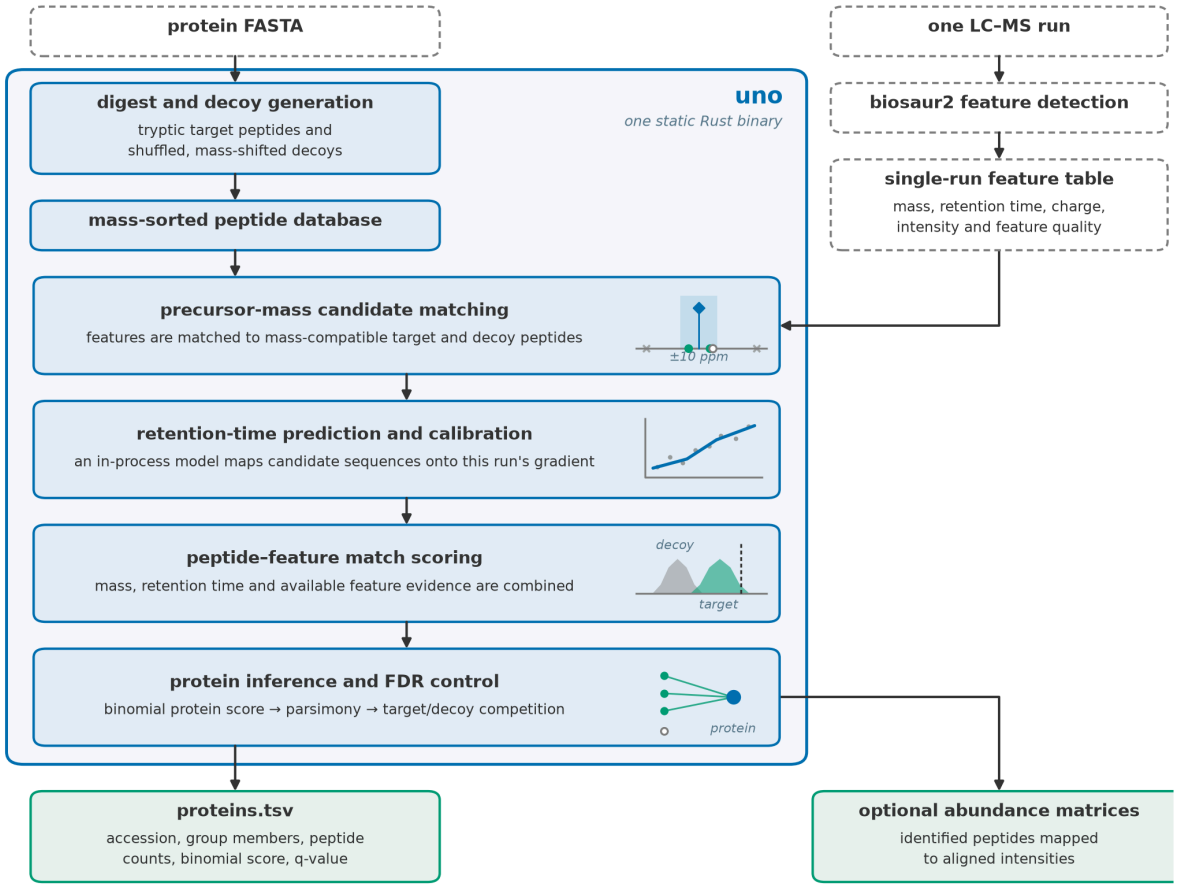
In-silico tryptic digestion is performed with sage-core[9] (enzyme rules, missed cleavages, length/mass bounds, fixed/variable modifications), and masses are recomputed in `f64`. Candidate lookup is a binary search over a mass-sorted peptide vector. Mass bounds are applied after modifications, which is the behavior an MS1-only search wants. Decoys are generated by interior-residue shuffling. The interior residues are pooled per protein rather than across the whole database, so each organism's residue composition is preserved and the per-protein binomial background stays an in-organism rate. Shuffling generally yields a decoy of different composition, and therefore of different mass, from its target, though nothing forbids a collision. A further mass shift is applied so that a decoy is not a mass twin of its target. We prefer these to same-mass reversed decoys because a reversed decoy is an RT-indistinguishable twin of its target under composition-based RT models (Supporting Information Section S9), and such twins give the protein-level null no room to separate. That argument is about composition-based predictors. uno's shipped model is order-aware and does recover most of the forward-minus-reverse signal, so the objection to reversed decoys is weaker for it, but the per-protein shuffle is what the reported searches used.

2.3 Feature–peptide matching

Each feature is matched to all peptides within a ppm tolerance, enumerating isotope-error offsets of zero and ± 1 carbon-13 neutron (1.00335 Da) on the neutral mass. The resulting peptide–feature matches (PFMs) are reduced to the top-K best-mass targets and the top-K best-mass decoys per feature, to bound the downstream cost without discarding the RT-correct assignment. K is 50 by default, so a feature carries at most 100 candidates into scoring.

2.4 Retention-time model

uno predicts RT in-process with a pure-Rust sequence model, so no Python or deep-learning runtime is required at search time. The model distributed with uno, and the one passed explicitly to every search reported in Section 3, is a convolutional network trained directly on experimental retention times from the Chronologer peptide library[10]. A learned residue embedding feeds a



Solid blue boxes inside the outlined panel are stages of the single uno binary; dashed boxes are inputs and external tools; green boxes are outputs.
For the benchmark, biosaur2 performs feature detection and both engines receive the identical single-run feature table.

Figure 1: uno's identification cascade. The two lanes at the top are its inputs: the FASTA is digested to a mass-sorted peptide database with decoys (left), and one run's feature file is read directly (right). Each feature is matched to mass-compatible peptides and their decoys, and each match is scored on recalibrated mass error and calibrated RT deviation. Identity is then concentrated at the protein level by a binomial survival score, greedy parsimony and a target/decoy FDR walk. The quantitation path, which reads a cross-run aligned intensity matrix instead of a feature file, is described in the Experimental Section and is not drawn here. Only the identification path is compared against ms1searchpy in this work, and it takes the same single-run input that ms1searchpy takes.

residual dilated convolution stack, a masked mean and max pool, and an MLP head that also receives 11 global physicochemical descriptors. A second, lighter option is a student model distilled from DeepLC predictions over a 99-dimensional order-aware feature encoding, with a multilayer-perceptron and a gradient-boosted[11] backend. Its characterization, together with a reverse-sequence benchmark that measures how much sequence-order information an RT model actually captures, is in Supporting Information Section S9. Both models predict RT for all candidate peptides in-process.

Because a frozen predictor lives in an abstract RT space, uno calibrates it to the run's observed gradient with a RANSAC-seeded[12], monotone piecewise-linear bootstrap on confident identifications, progressively tightened across rounds and vouched by protein-level multiplicity.

2.5 Scoring and FDR

PFMs are scored by a deviation discriminant combining recalibrated mass error (in σ units, via sigma-clipped re-estimation of the mass-error distribution) and calibrated RT deviation. Because mass and RT alone do not separate target from decoy at the single-peptide level in this regime (Section 3.4), identity is concentrated at the protein level. A binomial survival score $-\log_{10} P(X \geq k \mid n, p)$ evaluates whether a protein's k observed peptides exceed the random-match background rate p estimated from decoys, followed by greedy parsimony, target/decoy pair competition, and a target/decoy FDR walk. The protein FDR is the decoy-over-target ratio in its $(d+1)/t$ form, made monotone by a running minimum from the deepest rank upward. ms1searchpy reports its own protein list under the same convention, so the two are directly comparable.

Acceptance is set by sweeping a single combined deviation threshold over a fixed grid. The swept quantity is the sum of the squared mass error in σ units and the squared calibrated RT deviation, and the cutoff kept is the one accepting the most target proteins at the chosen FDR. The mass gate and the RT scale are not swept. They come from the monotone recalibration described above. This selection is an argmax over a noisy decoy-derived estimate, so the estimate at the selected cutoff is optimistically biased, in the same way any best-of-n order statistic is. Two things bound that bias. The sweep is capped so it cannot run away to a loose, decoy-heavy cutoff, and the entrapment probe of Section 3.3 measures the residual optimism directly rather than assuming it away.

A learned per-match rescorer runs by default ahead of the protein stage. It is a cross-validated gradient-boosted-tree classifier over the per-match features, including the feature finder's quality columns, trained on confident targets against decoys with folds assigned by peptide. Its ranking is then used to build a cumulative-inclusion ensemble: the protein binomial is evaluated over a series of progressively looser inclusion bands and the scores are averaged over the tightest of them. The stage is guarded, so it is kept only when it accepts more targets at the same estimated FDR than the deviation path alone, and it no-ops when the quality columns are absent. The classifier is pure Rust[13] and seeded, so the stage does not reintroduce a Python or deep-learning dependency and does not compromise determinism. This peptide level learning is distinct from protein-level rescoring, which is the subject of Section 3.4: uno's protein score is always the transparent binomial, and the learned LDA protein rescorer evaluated there is not part of the released engine.

All stages are implemented to be deterministic. No result-affecting path uses an unseeded RNG, and every path where hash-map iteration order could reach the result was made order-independent, so repeated runs on identical input produce byte-identical output. A two-run byte-diff on a full feature file is part of the test suite, alongside unit tests for the individual reductions (Section 3.2).

2.6 Quantitation

For multi-sample experiments, identified peptides and proteins are mapped onto koth's per-sample aligned intensities to emit peptide- and protein-level quant matrices. uno does not implement its own feature extraction. Where it reads raw data it embeds koth's finder, and every result reported here was computed from precomputed feature files. koth owns alignment, XIC integration, and match-between-runs. The quantitation path competes each aligned feature

to a single protein and groups indistinguishable proteins. It can also reassign a feature to the protein whose across-sample profile it tracks, which is an opt-in flag and was enabled for the results reported here. A minimum-runs floor is required: a precursor must be present in at least a stated number of samples to enter the search, without which the random-match background rises to the point that no protein is accepted.

Two per-cell intensity estimators are available in koth at the alignment stage. A box-sum takes all signal in the (m/z, RT) extraction window. An averagine projection instead reports the observed isotopologue vector projected onto the mass-derived averagine isotope pattern[14]. That projection acts as a matched filter, retaining on-pattern signal and rejecting co-eluting off-pattern interference. The interference-rejection property is a consequence of the projection rather than a result of the cited work. We use the averagine projection for quantitation because it is mass-based and identity-free, with no reliance on the MS1 peptide-level FDR, and because under the current koth feature finder it preserves the entrapment-scored protein-ID count where the box-sum regresses it. It is not koth's default and was enabled explicitly.

uno emits precursor- and protein-group-level abundance matrices and stops there. Downstream differential testing is left to standard tools, keeping uno an identification-and-quantitation engine rather than a statistics framework. The test used in Section 3.5 is a per-protein t-test on peptide fold changes with a fold-change gate and Benjamini–Hochberg control[15]. It follows DirectMS1Quant[16] in spirit rather than in detail, and Supporting Information Section S7 states where the two differ. The contrast is one-directional by construction: a precursor enters the test only if it is detected in at least two replicates of the more concentrated condition. That matches a spike-down dilution series, in which nothing is expected to rise as the spike is diluted, and it means the reported calls are not a two-sided result.

2.7 Benchmark data and comparison

The head-to-head benchmark uses six runs from the DirectMS1 papers' own deposits. Five are 5-minute HeLa runs from PXD015669 (Q Exactive HF, no FAIMS), referred to here by their run numbers 09746, 09766, 09734, 09778 and 09950. The sixth is LUM2_05433_VG from PXD023977 (Orbitrap Fusion Lumos with FAIMS). uno reads it as a single run and ignores its three compensation-voltage fractions. The feature count of each file is given in Table S3. Features were called by biosaur2 0.2.28[17] on mzML converted with ThermoRawFileParser 1.4.5[18], and the same feature file was handed to both engines.

Both engines searched one human plus *S. cerevisiae* FASTA (20,647 human and 6,066 yeast entries) and generated their own decoys internally. The digest was held identical (peptides of length 7 to 30, no missed cleavages, carbamidomethyl cysteine), so the comparison isolates the engines. uno was run once per dataset because it is deterministic, ms1searchpy 2.8.12 three times because it is not, always with `-deeplc 1 -ad 1` so that its RT model is DeepLC 3.1.13 and it generates decoys. Runs were serialized on one workstation, and wall-clock time, user CPU time and peak resident memory were captured with `/usr/bin/time -v`. Exact commands, versions and configuration files are in Section S1.

run10 and deephela were development data, not independent benchmarks. Four defaults (`top_k = 50`, the ensemble inclusion band, `min_child_weight = 80`, and `cut_max = 8.0`) were selected during development by maximizing yeast-entrapment depth on those related data. The six runs used for the primary head-to-head were not used to select among the reported configurations.

The shipped RT model was retained despite a lower run10 score than an alternative, rather than selected to maximize that tuning result.

The comparator was a locally patched ms1searchpy development state based on the 2.8 series, not an unmodified PyPI release. The patch lets ms1searchpy read the same five per-feature quality columns available to uno; stock ms1searchpy ignores them. This gives the comparator additional information and therefore makes the comparison conservative for uno. The exact commit and patch are supplied with the archived benchmark materials (Section S1).

Depth is reported by two-proteome entrapment rather than by either engine's own target/decoy estimate, using one scorer for both engines (`benchmark/entrapment.py`). The sample is human, yeast is absent from it, so any accepted yeast protein is a false positive. Ranking target proteins by score, let $n_{\text{human}}(k)$ and $n_{\text{yeast}}(k)$ be the accepted counts of each species at rank k, and N_{human} and N_{yeast} the number of entries of each in the searched database. The scale-corrected false-discovery proportion at rank k is

$$\mathrm{FDP}(k) = \frac{n_{\text{yeast}}(k)}{n_{\text{human}}(k)} \cdot \frac{N_{\text{human}}}{N_{\text{yeast}}} \tag{1}$$

and the running minimum of this from the deepest cutoff upward gives q. Depth at a threshold is the largest number of human proteins whose entrapment q meets it. Every depth number in this paper, for both engines, is an entrapment depth in this sense, not a count taken from either engine's own reported q. The two are compared directly in Section 3.3, where each engine's own reported list is scored against the same probe to check its calibration.

The database-size correction assumes that the entrapment and target proteomes are exchangeable apart from their size, so estimators of this family give a bound rather than an unbiased estimate[19,20]. Scaling by entries rather than by tryptic peptides is the conservative direction for the human counts reported here, and the Limitations return to how coarse the resulting bound is.

A second, independent comparison uses the three replicate 200 ng HeLa Lumos-with-FAIMS runs of PXD023977 (LUM2_05433, 05434 and 05435) with a different probe. There both engines searched a target-only human plus *E. coli* FASTA (20,193 human and 4,431 *E. coli* entries) built by stripping the deposit's premade decoy entries, and *E. coli* is the absent entrapment species. The replicate close-up of Section 3.2 and the quality-column ablation of Supporting Information Section S6 use a separate 211,064-feature medium-length HeLa run, referred to as run10, which carries the same quality columns.

External instrument validation uses three 10 ng Pierce HeLa runs from PXD066701, acquired in the Kelly laboratory on an Orbitrap Astral with FAIMS Pro at a fixed −45 V compensation voltage and an 11.4-minute dual-column cycle. The three raw files were selected as a complete matched set from the deposit metadata before either search result was examined. ThermoRawFileParser 1.4.5 converted the raw files and biosaur2 0.2.28 detected 85,274 to 90,376 features without skipping an MS1 scan. Both engines received each same feature table, human-plus-yeast FASTA, and digest used in the primary benchmark. uno was run once per file and ms1searchpy ten times per file; all searches were serialized. This dataset was not used for method development, parameter selection, or RT-model selection.

Quantitation is evaluated on the DirectMS1Quant[16] UPS1-into-*E. coli* dilution series (PXD033350), eight spike levels from 0.1 to 50 fmol × 3 replicates, 24 runs on an Orbitrap Fusion Lumos with a FAIMS Pro interface. It was searched against the 48 UPS1 spike proteins,

the *E. coli* proteome as a constant background, and the human proteome as a known-absent entrapment probe. Runs were converted, feature-found and cross-run-aligned by koth. uno searched the aligned matrix, and peptide fold-change differential testing was scored against the published DirectMS1Quant results.

The RT-model experiments of Supporting Information Section S9 and the LDA rescorer experiment of Section 3.4 predate this benchmark and were measured on an earlier development configuration, on run QEHF1_09744_JB of PXD015669 with koth-called features and the distilled MLP RT model. Their scope is stated where they are reported.

3 Results and Discussion

3.1 uno is deeper than ms1searchpy on identical input, on all six benchmark datasets

On the six DirectMS1-paper datasets, given the same feature file, the same database and the same digest, uno identified more proteins at 1% entrapment FDR than the median ms1searchpy replicate on every dataset (Figure 2). The margin was 3.0% on the shallowest run and 39.7% on the FAIMS run. At 5% entrapment FDR uno was deeper on five datasets and tied on the deepest of the 5-minute runs. Depth ranged from 862 proteins on the sparsest 5-minute run to 2072 on the FAIMS run, which reproduces the 2000-protein figure the reference method reports for that acquisition[5].

Against ms1searchpy's best replicate rather than its median, a harsher test, uno still wins three of the six. Which engine appears deeper on the other three depends on which ms1searchpy run one happens to keep, which is the subject of Section 3.2.

uno was also several times cheaper. Each search finished in 32 to 45 s of wall time against 195 to 247 s for ms1searchpy. That is a factor of 5.3 to 7.0. It also ran in roughly half the peak resident memory, and user CPU time separated the two engines in the same way. The gap holds on a larger input. On the run10 dataset uno identified 1309 proteins at 1% against an ms1searchpy median of 1155 (Table S7). It did so in 46 s against 360 s and in 3.8 GiB against 9.3 GiB. Per-dataset resource scaling is plotted in Figure S3 and tabulated in Table S3.

The result carries to the acquisition the reference method's 2000-protein claim[5] rests on. Because the FAIMS run in Figure 2 is one file scored against a yeast probe, we repeated the comparison on all three replicate 200 ng HeLa FAIMS runs of PXD023977 with a different and independent probe, the deposit's own human plus *E. coli* database with *E. coli* as the absent species. Averaged over the three runs, uno accepted 1591 proteins at 1% entrapment FDR against ms1searchpy's 1474, an 8% margin, and the margin at 5% was 7.6% (Figure S2, Table S5). ms1searchpy is the deeper engine on one replicate at 1% and on a different replicate at 5%. uno leads the remaining four of the six replicate-by-threshold comparisons and both means, and the spread across ms1searchpy's three replicates again exceeds the between-engine difference. uno reaches this depth while ignoring the compensation-voltage structure of the files entirely, treating the three CV fractions as one run, so the third dimension FAIMS provides remains unexploited upside.

Independent Orbitrap Astral validation.

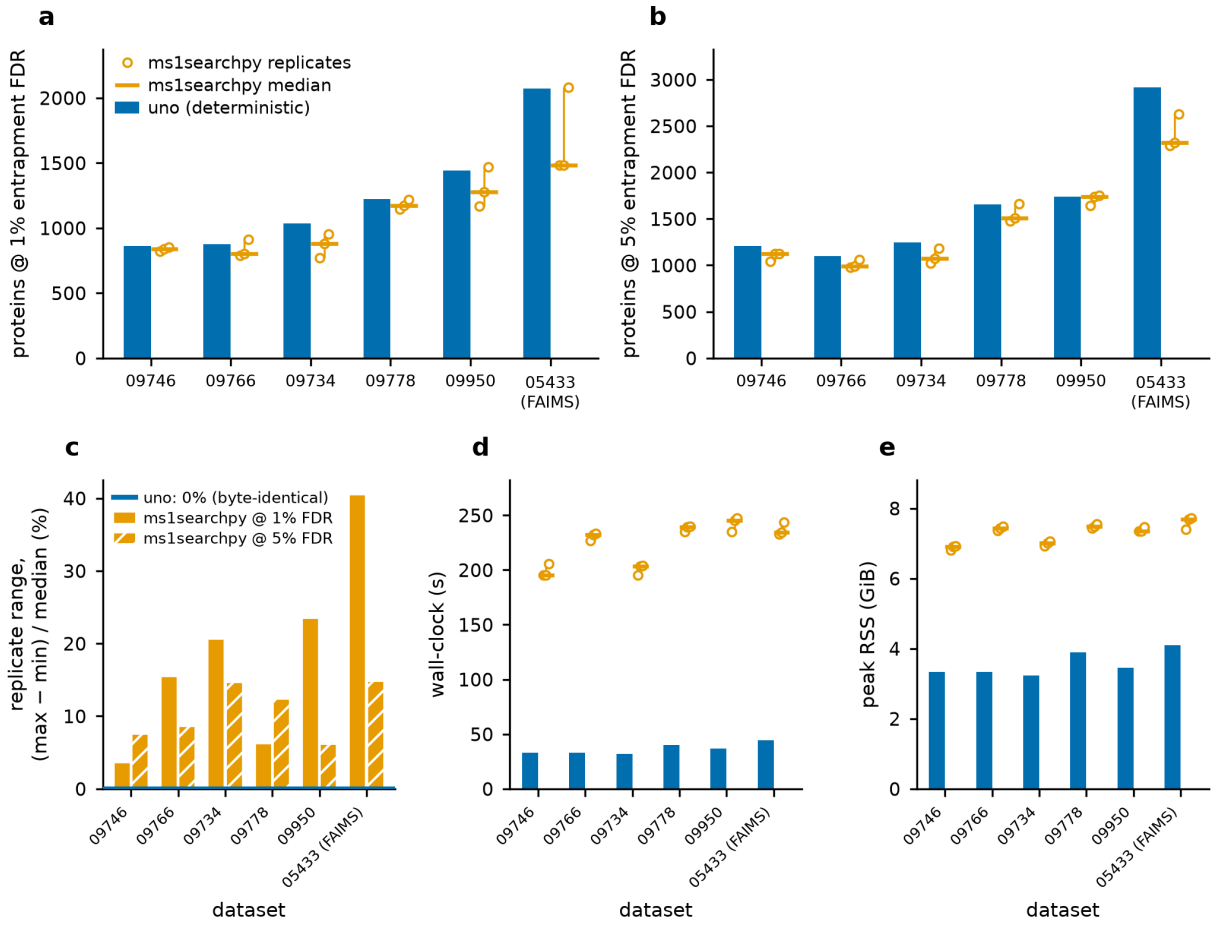


Figure 2: uno against ms1searchpy on six datasets from the DirectMS1 deposits, five 5-minute HeLa runs from PXD015669 and one FAIMS run from PXD023977, with the same single-run feature file, database and digest given to both engines. **(a)** proteins accepted at 1% two-proteome entrapment FDR and **(b)** at 5%. **(c)** each dataset's ms1searchpy replicate range as a percentage of its own median; uno's is 0% by construction, because identical reruns are byte-identical (Section 3.2). **(d)** wall-clock seconds and **(e)** peak resident memory. Bars are uno, which is deterministic and was run once. Points are the three ms1searchpy replicates with their median marked. uno is deeper at 1% on all six datasets and at 5% on five, and finishes 5.3 to 7.0 times faster in roughly half the memory.

The external Astral set tests whether the result extends beyond the instruments, laboratory, and deposits used above. At 1% entrapment FDR, uno accepted 1448, 1182, and 1006 human proteins across the three files (Figure 3). The corresponding ms1searchpy medians over ten searches were 45, 47, and 46, with per-file ranges of 26–110, 30–105, and 22–91. At 5%, uno accepted 2145, 1685, and 1567 proteins, against ms1searchpy medians of 45, 47, and 74.5. Thus the separation is not a favorable draw from the comparator's stochastic output.

The shallow comparator result reflects calibration on this acquisition rather than a failed search. ms1searchpy completed every run, but its initial stage supported only 5 to 14 proteins in representative runs and its calibrated RT standard deviation was 1.7 to 2.3 minutes, a large fraction of the 11.4-minute cycle. Removing the two optional isotope-quality columns gave 36 proteins at both thresholds on the first file, and using ms1searchpy's default RT model instead of DeepLC gave 26. Neither control changed the order of magnitude. uno also finished 9.5 to 10.1 times faster and used 2.0 to 2.5 times less peak memory. These data establish transfer

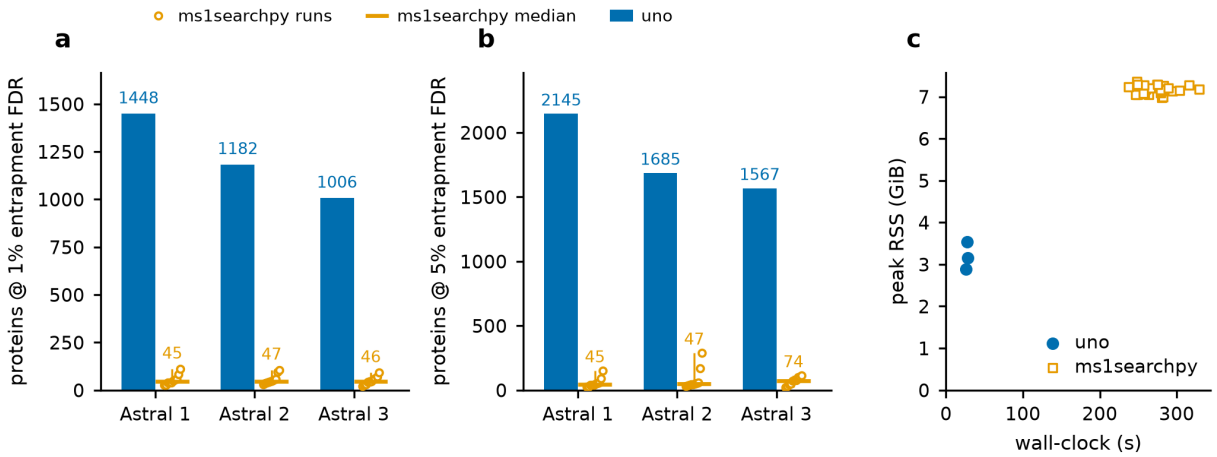


Figure 3: Independent Orbitrap Astral validation on three 10 ng HeLa runs from PXD066701. Both engines received the same biosaur2 feature table, database, and digest. **(a,b)** Human proteins accepted at 1% and 5% two-proteome entrapment FDR. Bars are one deterministic uno search per raw file; open circles are ten serialized ms1searchpy searches and horizontal ticks mark their medians. **(c)** Wall-clock time versus peak resident memory for the same searches. The comparison applies to this short-cycle, fixed-FAIMS-CV workflow and is not a claim about all Orbitrap Astral acquisitions.

to this short-cycle, fixed-CV Astral workflow; they do not imply the same margin for every Astral acquisition.

3.2 Reproducibility: uno is byte-identical across runs, ms1searchpy is not

Repeating a search on the same file should return the same proteins. It does not for the reference engine. Across the six datasets the three ms1searchpy replicates spanned ranges from 30 to 600 proteins, equal to 3.6% and 40.5% of the respective medians (Figure 2 c, Table S3, Figure S6). On run10, the tuning set, the same three-replicate protocol spanned 1079 to 1220 at 1% and 1534 to 1941 at 5%. The spread is not a property of the data. Input, database and parameters were byte-identical between replicates, and the variability comes from the engine's random decoy shuffle and its unseeded hyperparameter search. A single reported number from such an engine is a draw, not a measurement, which is why every ms1searchpy value in this paper is a median over three runs with its range shown.

uno returns the same protein list every time, by construction rather than by luck. Every source of run-to-run nondeterminism found in development traced to one cause, Rust's process-randomized hash-map iteration order, reaching the result through one of two routes. Either a floating-point reduction saw its sample in a different order, which is not associative and moves the low bits of a threshold, or a first-seen tie-break resolved differently. Each affected path was made order-independent by sorting on value before reducing, or on accession where a tie genuinely had to be broken; the six affected paths are listed in Supporting Information Section S8.

One of them is worth stating because it shows what this class of defect looks like. Two runs agreed on every one of 3325 proteins, on every score, peptide count and q-value, and disagreed on the reported group membership of 38 of them, because the protein-group table was keyed on accession alone and uno's decoy peptides carry the accessions of the targets they were shuffled

from. Depth was never affected, so no summary statistic would have exposed it. A byte-diff of two runs on one feature file did, and that check is now part of the test suite. After these fixes, independent runs on identical input are byte-identical. Determinism is thus a guaranteed property of the implementation, not a statistical observation, and it is what makes the single-run uno column of Figure 2 a fair comparator for a median of three.

3.3 Two-proteome entrapment calibrates the reported protein FDR

Target/decoy FDR can be optimistic if the decoys do not faithfully model the random-coincidence behavior of real-but-absent peptides. The entrapment probe answers two different questions, and we keep them apart throughout. The first is depth. Every depth figure in this paper, for both engines, is an entrapment depth, taken against the entrapment q as Methods defines it. The second is calibration. Take the protein list an engine itself reports at 1% and ask what the probe says its false-discovery proportion actually is. The two questions have different answers on the same search. On one 5-minute run, uno's own 1% list holds 830 human proteins while 1035 clear the entrapment criterion at the same threshold (Table S4).

On calibration, uno's reported 1% list carried an empirical FDP between 0.00% and 2.16%, a median of 0.72% (Table S4, Figure 4). Applying the nominal threshold literally, uno is conservative on four of the six datasets. It sits at the nominal level on 09778, at 1.03%, and is about twice optimistic on 09746, at 2.16%. The reference engine exceeded its nominal 1% in 12 of its 18 runs, with a worst case of 2.07% and a median across datasets of 1.20%. No single ms1searchpy run was as optimistic as uno's worst dataset, but that is a comparison of extremes. Run for run, ms1searchpy's empirical FDP exceeded uno's on 13 of the 18 pairings. The reference engine is therefore the more often optimistic of the two, and uno the more severely optimistic in its single worst case. At a reported 5% two uno datasets again exceed the threshold, 09746 at 6.65% and 09766 at 5.17%, and the other four sit between 2.46% and 3.61%.

Replicate spread confounds the reference engine's calibration as much as its depth. On one 5-minute run the three ms1searchpy replicates put the empirical FDP between 0.39% and 1.62% on identical input, a range as wide as the difference between the two engines. The depth axis carries the same warning. On run10, ms1searchpy's internal 1% count was 1319 proteins but only 1220 of those survive the entrapment criterion. Single-dataset FDP values are quantized and noisy, because an accepted set contains only a handful of entrapment proteins (Section S3), so the pooled median is the statistic to read rather than any individual panel.

3.4 Entrapment supports the transparent binomial protein score

In this regime mass and RT cannot separate target from decoy peptides. Among matches with near-zero RT deviation (< 0.01 min), target and decoy peptides occur at approximately 50:50. Most target peptides in a proteome are not actually present in a given sample, so a random target match is no more likely than a decoy match. Discrimination arises only from multiplicity. Real, present proteins accumulate several RT-consistent peptides while decoy proteins remain singletons, which is what the binomial protein model exploits.

The feature finder's quality columns carry little target/decoy information in a run10 ablation, because targets and decoys both match real, well-formed features by mass coincidence. Retention time is more discriminating, but protein-level multiplicity remains the main identification signal (Supporting Information Section S6).

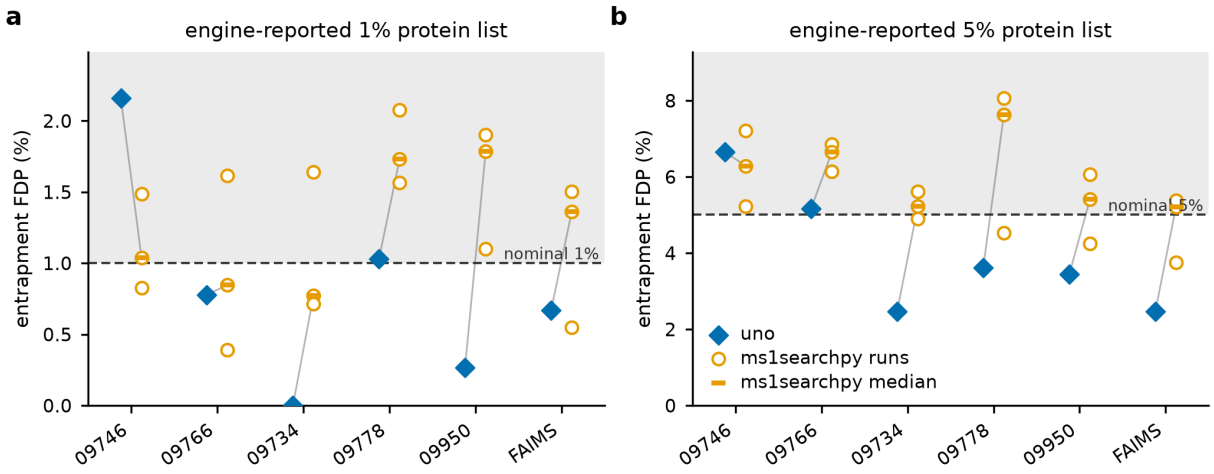


Figure 4: Protein-level FDR calibration under two-proteome entrapment, human sample searched against human plus *S. cerevisiae*. Entrapment FDP is shown for each engine's own reported **(a)** 1% and **(b)** 5% protein list. Diamonds are uno's deterministic result. Open circles are the three ms1searchpy runs and orange ticks are their medians. The dashed horizontal line is the nominal operating point; values above it are optimistic. Full calibration curves are in Figure S1.

A protein-level learner was therefore an important negative control. On one earlier development configuration, a cross-validated Fisher LDA increased the human-only count but accepted seven entrapment proteins where the binomial accepted one. Additional regularization did not restore calibration. This result does not establish that every learned protein scorer must fail; it shows that the tested scorer's apparent gain did not survive external validation. uno therefore retains the binomial as its released protein score. The model, safeguards, counts, and development-configuration caveat are reported in Supporting Information Section S9.

3.5 Quantitation tracks a spike-in dilution

On the DirectMS1Quant UPS1-into-*E. coli* dilution series, uno identifies 326 proteins at 1% FDR, at an entrapment FDP of 0.34%, and recovers the designed structure of the experiment (Figure 5). Protein abundances are normalized to each protein's value at 50 fmol, so the curves share that reference point; missing-cell handling and aggregation are specified in Supporting Information Section S7. The primary quantitative result is therefore the trend across the remaining concentrations. UPS1 protein abundance falls with spike amount at a median Spearman ρ of +0.71 against log spike amount, and 89% of the 18 quantified UPS1 proteins sit above $\rho = 0.5$. The constant *E. coli* background and the quantified human probe stay flat, at median ρ of +0.02 and +0.25 (Table S8). The spike and constant background are well separated. The five quantified human probe proteins provide only a small negative-control sample and are not treated as a general distributional result.

Differential calling at the dilute end trails the reference. On the extreme 50 against 0.1 fmol contrast, DirectMS1Quant[16] reports 42 changed UPS proteins with no false positives, while a peptide-fold-change test on uno's matrix, scored under a stricter protocol with a human entrapment probe present, recovers 22 at 21% precision (Supporting Information Section S7). The result does not validate trace-level differential testing. Detection loss at the dilute end is a major contributor: short peptide traces are discarded, while alignment can fill runs where the peptide was not independently detected. Supporting Information Section S7 gives the detection

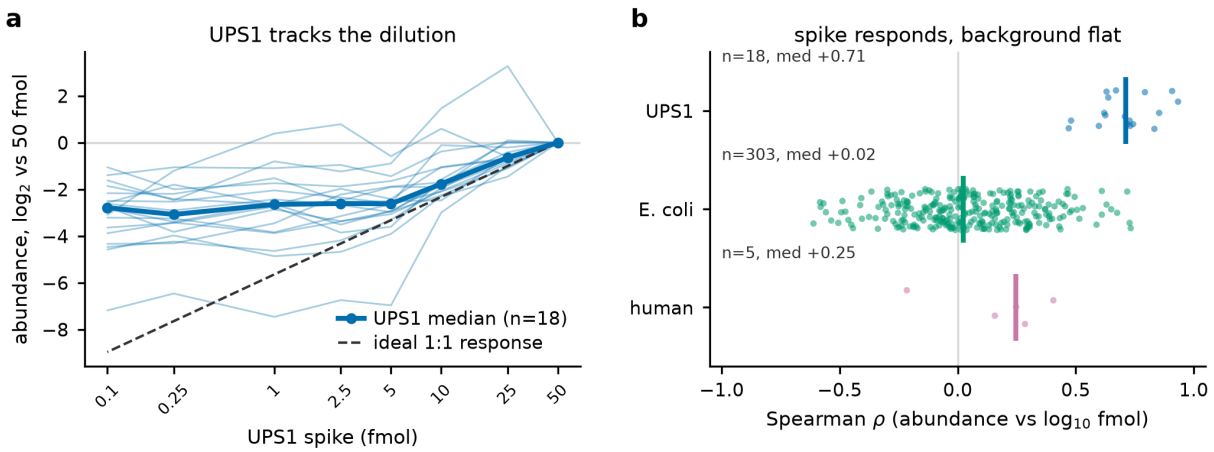


Figure 5: Spike-in dilution response on PXD033350, eight UPS1 levels from 0.1 to 50 fmol into a constant *E. coli* background, three replicates each, with the human proteome searched as a known-absent probe. **(a)** UPS1 protein abundance against spike amount. **(b)** the distribution of per-protein Spearman correlations against log spike amount, by species. The spike tracks the dilution while the constant background stays flat; only five human probe proteins meet the quantitation filter. The fold-change contrast at the dilute end is limited by detection and is reported in Supporting Information Section S7.

traces, threshold ablation, and imputation sensitivity. The experiment supports identification-linked dilution tracking only.

4 Discussion

The principal result is not identification count alone, but greater depth under one external entrapment criterion applied to both engines. This same-input design holds feature detection, database, and digest fixed, while entrapment prevents an engine's own target/decoy estimate from defining success. Across the tested short gradient runs, uno combines that depth with substantially lower resource use and exact repeatability.

These findings establish a practical advantage within the benchmarked settings, not universal superiority across proteomics workflows. The independent Astral set broadens the instrument and laboratory coverage, but all identification benchmarks remain short-gradient HeLa acquisitions. uno does not use the FAIMS compensation-voltage structure, and the entrapment estimate is quantized by a small number of accepted absent-species proteins. The abundance experiment adds a narrower conclusion: uno-linked protein abundance follows a designed dilution above the trace range, but the present pipeline does not validate differential testing at the dilute endpoint.

Determinism matters beyond convenience. A byte-identical result can be audited, regression-tested, and compared across software releases without conflating a code change with a random draw. Together with external FDR calibration, this is the main scientific and operational advance of uno.

5 Limitations

These limitations delimit where MS1-only search and uno apply. The first is inherent to the data rather than to uno. Mass and RT cannot separate target from decoy at the single-peptide

level (Section 3.4), so identification depends on protein-level multiplicity, and any per-peptide discriminant is bounded by that.

Second, on Orbitrap data without ion mobility, peptide-centric extraction from raw hills did not add identifications on the earlier development configuration where we tested it, because the (m/z, RT) coordinate is too degenerate without a third dimension. Ion-mobility data should lift this. A distilled collisional-cross-section model is trained and loadable, but it is not yet wired into scoring, so uno does not currently use mobility as a discriminant and the benchmarks here do not test it. The FAIMS runs are read without their compensation-voltage structure, so that dimension is likewise unexploited.

Third, for a single enzyme and a single run, expanding search breadth is counterproductive. Adding missed cleavages or modifications inflates the theoretical count faster than the observed count and raises the decoy background, reducing identifications, consistent with the minimal digest of DirectMS1[5]. This is a statement about breadth within one specificity. Breadth across complementary specificities is a different proposition, and multiple enzymes have been shown to help MS/MS-free identification[3].

Related to this, uno currently yields approximately zero identifications on TMT-labeled MS1 data. The blocker is uno's own retention-time encoder, which takes a stripped sequence and has no channel for a modification, so a TMT-labeled peptide is predicted at the retention time of its unlabeled form and calibration cannot seed. The difficulty is not specific to uno. Predicted retention time for TMT-modified peptides correlates only $r \approx 0.71$ with observed retention time even for DeepLC, which does model modifications.

Fourth, the entrapment probe bounds the empirical FDR only coarsely. One yeast acceptance is worth 0.17 to 0.41 percentage points of estimated FDP at the depths reached here, and an accepted set holds at most 8 of them, so a single-dataset estimate is quantized (Section S3). The benchmark is also narrow in acquisition terms: nine primary runs, two laboratories, one sample type (HeLa), three instrument models, and short gradients throughout. The Astral validation covers one fixed-CV acquisition design rather than the full range of Astral methods. Longer gradients are off-design for the method and were not benchmarked here.

Fifth, uno's quantitation is validated here for dilution tracking, which is robust, and not for differential testing, which is not. On the extreme contrast of the spike-in series uno recovers about half as many changed proteins as DirectMS1Quant[16] and does so at 21% precision, against a reference output with no false positives. Upstream feature detection is one identified contributor: trace peptides yield 1 to 2 scan hills that the feature finder discards. The aligned matrix compounds this. Alignment transfers a match into runs where the precursor was never independently found, so a filled cell at the dilute end is weaker evidence than a filled cell at the abundant end (Section 3.5). The differential test also runs over the full precursor matrix rather than over the FDR-controlled protein set. The scoped remedy is a targeted, identification-seeded matched-filter re-extraction rather than a global detection threshold, which trades specificity for recall.

Two of these bounds could be tightened directly. A size-matched entrapment probe, and the same probe applied across a wider dataset panel, would sharpen the absolute FDR estimate beyond the coarse bound above. A formal reproducibility experiment, at least ten runs of each engine on each dataset, would replace the three-replicate ranges of Section 3.2 with a proper variance estimate.

6 Conclusions

uno is a fast, deterministic, open-source Rust engine for MS1-only protein identification. Given the same single-run feature file that the reference implementation takes, it identified more proteins at 1% entrapment FDR on all six DirectMS1-paper datasets tested, ran 5.3 to 7.0 times faster, and used roughly half the peak memory. Repeating the comparison on the reference method's own three FAIMS replicates leaves uno ahead by 8% on average at 1% FDR, with ms1searchpy the deeper engine on one replicate of the three. On three independent 10 ng Orbitrap Astral runs, uno accepted 1006 to 1448 proteins at 1% entrapment FDR against ms1searchpy medians of 45 to 47 and ran approximately 10 times faster. It returns byte-identical output across repeated runs, while ms1searchpy's answer moved by up to 600 proteins. Its retention-time model runs in-process in pure Rust, so the Python and deep-learning dependency stack is gone. Two-proteome entrapment finds uno's reported protein FDR conservative on four of the six datasets, at the nominal level on one and about twice optimistic on one. A tested learned protein rescorer raised apparent counts but failed the same entrapment check, supporting uno's transparent binomial protein score. On a spike-in dilution series, uno-linked abundances track the design above the trace range; differential testing at the most dilute contrast remains outside the validated scope.

Associated Content

Supporting Information

The Supporting Information is available free of charge at [ACS DOI/URL, to be assigned on acceptance].

Benchmark protocol, software versions, hardware and the exact commands behind the identification, FAIMS and quantitation benchmarks; per-dataset and per-replicate identification depth, CPU time, wall-clock time and peak resident memory; entrapment calibration of each engine's own reported protein list, with per-dataset false-discovery-proportion curves; the FAIMS benchmark on all three replicate runs; resource scaling against input feature count; the per-feature quality-column ablation and per-column target/decoy separation; extended quantitation detail, including the differential contrast, per-species dilution tracking and the detection-loss traces; run-to-run reproducibility detail; the and development-configuration retention-time and rescorer experiments (PDF).

Author Information

Corresponding Author

Patrick Garrett, Department of Chemical Physiology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla, CA 92037, USA; Email: pgarrett@scripps.edu

Authors

John R. Yates III, Department of Chemical Physiology, The Scripps Research Institute, 10550 North Torrey Pines Road, La Jolla, CA 92037, USA

Author Contributions

P.G. conceived and implemented uno, designed and performed the benchmark experiments, analyzed the data, and drafted the manuscript. J.R.Y. supervised the project and revised the manuscript. Both authors reviewed and approved the final version.

Notes

The authors declare no competing financial interest.

Ethics

This study is a secondary computational analysis of publicly deposited mass spectrometry data (PXD015669, PXD023977, PXD033350 and PXD066701). The samples are HeLa, an immortalized cell line, and commercial standards, not material derived from identifiable human subjects or from any animal procedure. It involved no new sample collection and was determined to require no additional ethics review.

Acknowledgments

This work was supported by the U.S. National Institutes of Health (grants R01 HL165168, R01 AG077046, R01 MH100175, and R01 AG075862 to J.R.Y.; R01 MH132570 to L. Ye; and U01 AG088679 to S. A. Lipton) and by the Skaggs Graduate School of Chemical and Biological Sciences at The Scripps Research Institute (P.G.). During the preparation of this work the authors used Anthropic Claude large language models (via the Claude Code interface) for manuscript drafting and editing, for software-development assistance, and for a pre-submission audit of the manuscript against its own source data; no AI tools were used to generate figures or to produce, analyze, or alter any experimental data. The authors reviewed and edited all content as needed and take full responsibility for the content of the publication.

Data and Software Availability

Data. The mass spectrometry data reanalyzed here are publicly available from the ProteomeXchange Consortium via the PRIDE partner repository under accessions PXD015669 (5-minute HeLa, Q Exactive HF), PXD023977 (200 ng HeLa, Orbitrap Fusion Lumos with FAIMS) and PXD033350 (UPS1-into-*E. coli* dilution series, Orbitrap Fusion Lumos with FAIMS Pro), and PXD066701 (10 ng HeLa, Orbitrap Astral with FAIMS Pro).

Code. uno is open source (MIT licensed) at [repository URL, to be assigned on release]. The exact state used for this paper, together with the benchmark suite, the analysis and figure scripts, the configuration files and the manuscript source, will be frozen on a `paper` branch so that the figures and tables can be regenerated as published, and archived at Zenodo (DOI: [10.5281/zenodo.XXXXXXX, to be assigned on acceptance]). The retention-time distillation labels and the trained retention-time models are included in the same archive. The koth feature finder and cross-run aligner, used only by the quantitation path of Section 3.5, is archived alongside it.

References

(1) Zimmer, J. S. D.; Monroe, M. E.; Qian, W.-J.; Smith, R. D. Advances in Proteomics Data Analysis and Display Using an Accurate Mass and Time Tag Approach. *Mass Spectrometry Reviews* **2006**, *25* (3), 450–482. https://doi.org/10.1002/mas.20071.

(2) Ivanov, M. V.; Bubis, J. A.; Gorshkov, V.; Tarasova, I. A.; Levitsky, L. I.; Lobas, A. A.; Solovyeva, E. M.; Pridatchenko, M. L.; Kjeldsen, F.; Gorshkov, M. V. DirectMS1: MS/MS-Free Identification of 1000 Proteins of Cellular Proteomes in 5 Minutes. *Analytical Chemistry* **2020**, *92* (6), 4326–4333. https://doi.org/10.1021/acs.analchem.9b05095.

(3) Ivanov, M. V.; Tarasova, I. A.; Levitsky, L. I.; Solovyeva, E. M.; Pridatchenko, M. L.; Lobas, A. A.; Bubis, J. A.; Gorshkov, M. V. MS/MS-Free Protein Identification in Complex Mixtures Using Multiple Enzymes with Complementary Specificity. *Journal of Proteome Research* **2017**, *16* (11), 3989–3999. https://doi.org/10.1021/acs.jproteome.7b00365.

(4) Xiao, Z.; Tüshaus, J.; Kuster, B.; The, M.; Wilhelm, M. SWAPS: A Modular Deep-Learning Empowered Peptide Identity Propagation Framework Beyond Match-between-Run. *Journal of Proteome Research* **2025**, *24* (4), 1926–1940. https://doi.org/10.1021/acs.jproteome.4c00972.

(5) Ivanov, M. V.; Bubis, J. A.; Gorshkov, V.; Abdrakhimov, D. A.; Kjeldsen, F.; Gorshkov, M. V. Boosting MS1-Only Proteomics with Machine Learning Allows 2000 Protein Identifications in Single-Shot Human Proteome Analysis Using 5 Min HPLC Gradient. *Journal of Proteome Research* **2021**, *20* (4), 1864–1873. https://doi.org/10.1021/acs.jproteome.0c00863.

(6) Bouwmeester, R.; Gabriels, R.; Hulstaert, N.; Martens, L.; Degroeve, S. DeepLC Can Predict Retention Times for Peptides That Carry as-yet Unseen Modifications. *Nature Methods* **2021**, *18* (11), 1363–1369. https://doi.org/10.1038/s41592-021-01301-5.

(7) Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*; 2017.

(8) Garrett, P.; Yates, J. R., III. Koth: A Rust Feature Finder and Cross-Run Label-Free Quantification Pipeline for LC–MS Proteomics, 2026.

(9) Lazear, M. R. Sage: An Open-Source Tool for Fast Proteomics Searching and Quantification at Scale. *Journal of Proteome Research* **2023**, *22* (11), 3652–3659. https://doi.org/10.1021/acs.jproteome.3c00486.

(10) Wilburn, D. B.; Shannon, A. E.; Spicer, V.; Richards, A. L.; Yeung, D.; Swaney, D. L.; Krokhin, O. V.; Searle, B. C. Deep Learning from Harmonized Peptide Libraries Enables Retention Time Prediction of Diverse Post Translational Modifications. *bioRxiv* **2023**. https://doi.org/10.1101/2023.05.30.542978.

(11) gbdt-rs contributors. *gbdt-rs: An implementation of gradient boosting regression trees in Rust*. https://crates.io/crates/gbdt.

(12) Fischler, M. A.; Bolles, R. C. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM* **1981**, *24* (6), 381–395. https://doi.org/10.1145/358669.358692.

(13) Garrett, P. *sequoia-boost: A pure-Rust gradient-boosted decision tree library*. https://github.com/pgarrett-scripps/sequoia-boost.

(14) Senko, M. W.; Beu, S. C.; McLafferty, F. W. Determination of Monoisotopic Masses and Ion Populations for Large Biomolecules from Resolved Isotopic Distributions. *Journal of*

the American Society for Mass Spectrometry **1995**, *6* (4), 229–233. https://doi.org/10.1016/1044-0305(95)00017-8.

(15) Benjamini, Y.; Hochberg, Y. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* **1995**, *57* (1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x.

(16) Ivanov, M. V.; Bubis, J. A.; Gorshkov, V.; Tarasova, I. A.; Levitsky, L. I.; Solovyeva, E. M.; Lipatova, A. V.; Kjeldsen, F.; Gorshkov, M. V. DirectMS1Quant: Ultrafast Quantitative Proteomics with MS/MS-Free Mass Spectrometry. *Analytical Chemistry* **2022**, *94* (38), 13068–13075. https://doi.org/10.1021/acs.analchem.2c02255.

(17) Abdrakhimov, D. A.; Bubis, J. A.; Gorshkov, V.; Kjeldsen, F.; Gorshkov, M. V.; Ivanov, M. V. Biosaur: An Open-Source Python Software for Liquid Chromatography–Mass Spectrometry Peptide Feature Detection with Ion Mobility Support. *Rapid Communications in Mass Spectrometry* **2021**, e9045. https://doi.org/10.1002/rcm.9045.

(18) Hulstaert, N.; Shofstahl, J.; Sachsenberg, T.; Walzer, M.; Barsnes, H.; Martens, L.; Perez-Riverol, Y. ThermoRawFileParser: Modular, Scalable, And Cross-Platform RAW File Conversion. *Journal of Proteome Research* **2020**, *19* (1), 537–542. https://doi.org/10.1021/acs.jproteome.9b00328.

(19) Wen, B.; Freestone, J.; Riffle, M.; MacCoss, M. J.; Noble, W. S.; Keich, U. Assessment of False Discovery Rate Control in Tandem Mass Spectrometry Analysis Using Entrapment. *Nature Methods* **2025**, *22* (7), 1454–1463. https://doi.org/10.1038/s41592-025-02719-x.

(20) Madej, D.; Lam, H. On the Use of Tandem Mass Spectra Acquired from Samples of Evolutionarily Distant Organisms to Validate Methods for False Discovery Rate Estimation. *PROTEOMICS* **2024**, *24* (15), 2300398. https://doi.org/10.1002/pmic.202300398.

(21) Yu, F.; Haynes, S. E.; Nesvizhskii, A. I. IonQuant Enables Accurate and Sensitive Label-Free Quantification with FDR-Controlled Match-between-Runs. *Molecular & Cellular Proteomics* **2021**, *20*, 100077. https://doi.org/10.1016/j.mcpro.2021.100077.

(22) Kyte, J.; Doolittle, R. F. A Simple Method for Displaying the Hydropathic Character of a Protein. *Journal of Molecular Biology* **1982**, *157* (1), 105–132. https://doi.org/10.1016/0022-2836(82)90515-0.

(23) Krokhin, O. V. Sequence-Specific Retention Calculator. Algorithm for Peptide Retention Prediction in Ion-Pair RP-HPLC: Application to 300- and 100-Å Pore Size C18 Sorbents. *Analytical Chemistry* **2006**, *78* (22), 7785–7795. https://doi.org/10.1021/ac060777w.

(24) Eisenberg, D.; Schwarz, E.; Komaromy, M.; Wall, R. Analysis of Membrane and Surface Protein Sequences with the Hydrophobic Moment Plot. *Journal of Molecular Biology* **1984**, *179* (1), 125–142. https://doi.org/10.1016/0022-2836(84)90309-7.

(25) Klammer, A. A.; Yi, X.; MacCoss, M. J.; Noble, W. S. Improving Tandem Mass Spectrum Identification Using Peptide Retention Time Prediction across Diverse Chromatography Conditions. *Analytical Chemistry* **2007**, *79* (16), 6111–6118. https://doi.org/10.1021/ac070262k.

(26) Petritis, K.; Kangas, L. J.; Yan, B.; Monroe, M. E.; Strittmatter, E. F.; Qian, W.-J.; Adkins, J. N.; Moore, R. J.; Xu, Y.; Lipton, M. S.; Camp, D. G.; Smith, R. D. Improved Peptide Elution Time Prediction for Reversed-Phase Liquid Chromatography-MS by Incorporating Peptide Sequence Information. *Analytical Chemistry* **2006**, *78* (14), 5026–5039. https://doi.org/10.1021/ac060143p.

(27) Moruz, L.; Käll, L. Peptide Retention Time Prediction. *Mass Spectrometry Reviews* **2017**, *36* (5), 615–623. https://doi.org/10.1002/mas.21488.

(28) Käll, L.; Canterbury, J. D.; Weston, J.; Noble, W. S.; MacCoss, M. J. Semi-Supervised Learning for Peptide Identification from Shotgun Proteomics Datasets. *Nature Methods* **2007**, *4* (11), 923–925. https://doi.org/10.1038/nmeth1113.

(29) The, M.; MacCoss, M. J.; Noble, W. S.; Käll, L. Fast and Accurate Protein False Discovery Rates on Large-Scale Proteomics Data Sets with Percolator 3.0. *Journal of the American Society for Mass Spectrometry* **2016**, *27* (11), 1719–1727. https://doi.org/10.1007/s13361-016-1460-7.

Supporting Information

uno: A Fast, Reproducible, Dependency-Light Rust Engine for MS1-Only Protein Identification

Patrick Garrett, John R. Yates III

This section gives the commands, versions and configuration behind the identification, FAIMS and quantitation benchmarks of Sections 3.1 to 3.5. It then gives the full per-dataset and per-replicate results, extended entrapment and quantitation detail, the FAIMS and resource-scaling comparisons, the quality-column ablation and the run-to-run reproducibility detail. The retention-time and rescorer experiments of Section S9 were made on an earlier development configuration that predates the archived pipeline and cannot be re-run from it. Figures and tables are numbered S1, S2, and so on.

S1: Benchmark protocol, software versions and commands

Both engines received the identical single-run feature file, the identical FASTA and an identical digest specification on every dataset. uno was run once per dataset because its output is byte-identical between runs, ms1searchpy three times because its output is not. Runs were serialized on one workstation so that concurrent load could not corrupt the timings, and resource use was captured with `/usr/bin/time -v`. Depth was scored for both engines by the same entrapment script, described in the main text (Methods, Benchmark data and comparison). Software versions and the benchmark hardware are given in Table S1, and Table S2 maps each run's name in the main text onto its deposit and its feature file on disk.

Component	Version	Role
uno	0.1.0, commit `7c80199`	MS1-only search and quantitation
rustc	1.91.0	uno build (`cargo build --release`)
ms1searchpy	2.8.12	reference MS1-only engine
DeepLC	3.1.13	ms1searchpy retention-time model
LightGBM	4.6.0	ms1searchpy rescoring
pyteomics	4.7.5	ms1searchpy dependency
biosaur2	0.2.28	feature detection, both engines
koth	0.1.0, commit `eac09b7`	feature detection and cross-run alignment, quantitation only
ThermoRawFileParser	1.4.5	`.raw` to mzML conversion
Python	3.12.3	ms1searchpy environment, scoring scripts
NumPy / SciPy	1.26.4 / 1.16.2	scoring scripts

Component	Version	Role
CPU	Intel Core i7-12700H, 14 cores / 20 threads	all timings
Memory	62 GB	all peak-RSS figures

Table S1: Software and hardware used for every measurement reported here.

Name in text	Deposit	Raw file	Feature file (`benchmark/data/`)
09746	PXD015669	`..._09746_...raw`	`hela5min_09746.features.tsv`
09766	PXD015669	`..._09766_...raw`	`hela5min_09766.features.tsv`
09734	PXD015669	`..._09734_...raw`	`hela5min_09734.features.tsv`
09778	PXD015669	`..._09778_...raw`	`hela5min_09778.features.tsv`
09950	PXD015669	`..._09950_...raw`	`hela5min_09950.features.tsv`
LUM2_05433_VG	PXD023977	`LUM2_05433_VG.raw`	`faims_LUM2_05433_VG.features.tsv`
run10 (tuning set)	PXD015669	`..._run10_...raw`	`run10.features.tsv`
QEHF1_09744_JB	PXD015669	`QEHF1_09744_JB.raw`	`09744.features.tsv`
deephela	PXD004732	240-min HeLa	`deephela.features.tsv`

Table S2: Dataset names, deposits and local feature files. The six benchmark runs are above the rule. Below it are the three runs used elsewhere in this work and not part of the head-to-head: run10, which Section 3.2 reports and which is a tuning set rather than a benchmark; QEHF1_09744_JB, used for the development-configuration experiments of Sections 3.4 and S9, which is a different run from 09746 despite the similar number; and deephela, a 240-minute gradient used only where stated.

The commands below regenerate the identification, FAIMS and quantitation results.

```
# ------------------------------------------------------------------ variables
ROOT=$PWD                                   # the repository root
DS=hela5min_09746                           # one of the six (see Table S2)
OUT=$ROOT/benchmark/results/ms1_${DS}       # ms1searchpy writes here
RT=$ROOT/models/rt_cnn_chrono_global_v3_ch32.urtc
MS1=/path/to/ms1searchpy                    # the patched 2.8.12, see Section S1
mkdir -p "$OUT"

# 0. inputs: raw file -> mzML -> single-run biosaur2 features
#    (benchmark/README.md; run once per dataset, output named per Table S2)
docker run --rm -v $PWD:/data \
  quay.io/biocontainers/thermorawfileparser:1.4.5--ha8f3691_0 \
  ThermoRawFileParser -i /data/RUN.raw -o /data -f 2
biosaur2 RUN.mzML -o $ROOT/benchmark/data/${DS}.features.tsv

# 1. identification benchmark, both engines, identical input (benchmark/run.sh)
/usr/bin/time -v $ROOT/target/release/uno \
  --features $ROOT/benchmark/data/${DS}.features.tsv \
  --fasta    $ROOT/benchmark/data/entrap_human_yeast.fasta \
  --rt-model $RT \
  --config   $ROOT/benchmark/uno.toml \
  --out-proteins $ROOT/benchmark/results/uno_${DS}.tsv

# ms1searchpy writes beside its input, so it is run from $OUT on a copy.
```

```
cp $ROOT/benchmark/data/${DS}.features.tsv "$OUT/"
( cd "$OUT" && /usr/bin/time -v $MS1 ${DS}.features.tsv \
    -d $ROOT/benchmark/data/entrap_human_yeast.fasta -deeplc 1 -ad 1 )

# 2. entrapment scoring, identical scorer for both engines
python3 $ROOT/benchmark/entrapment.py \
  uno $ROOT/benchmark/results/uno_${DS}.tsv \
  ms1 $OUT/${DS}.features_proteins_full.tsv

# 3. FAIMS benchmark (benchmark/faims/run_faims.sh), E. coli entrapment
DATA=/path/to/PXD023977                     # the deposit, downloaded
FOUT=$ROOT/benchmark/faims/faims_out; mkdir -p "$FOUT"
R=05433                                     # replicate: 05433, 05434 or 05435

# Target-only DB: strip the deposit's pre-made DECOY_ entries, since both
# engines generate their own. The two counts set the entrapment scale factor.
awk '/^>/{p=(!/^>DECOY_/)} p' $DATA/sprot_human_and_ecoli_shuffled.fasta \
  > $FOUT/hye_target.fasta
export N_TARGET_DB=$(grep -c '_HUMAN' $FOUT/hye_target.fasta)
export N_ENTRAP_DB=$(grep -c '_ECOLI' $FOUT/hye_target.fasta)

$ROOT/target/release/uno --features $DATA/LUM2_${R}_VG.features.tsv \
  --fasta $FOUT/hye_target.fasta --rt-model $RT \
  --config $ROOT/benchmark/uno.toml --out-proteins $FOUT/uno_${R}.tsv
cp $DATA/LUM2_${R}_VG.features.tsv "$FOUT/"
( cd "$FOUT" && $MS1 LUM2_${R}_VG.features.tsv -d hye_target.fasta \
    -deeplc 1 -ad 1 )

cd $ROOT/benchmark/faims
python3 score_entrap.py uno $FOUT/uno_${R}.tsv
python3 score_entrap.py ms1 $FOUT/LUM2_${R}_VG.features_proteins_full.tsv

# 4. quantitation (benchmark/quant/README.md), PXD033350, 24 runs
QDATA=/path/to/PXD033350                    # the deposit, downloaded
QRES=$ROOT/benchmark/quant/results; mkdir -p "$QRES"

# 4a. per-run koth features, then cross-run alignment. averagine_projection is
#     NOT koth's default and must be set; the box-sum arm it replaces regresses
#     the entrapment-scored protein count (Section S7).
printf '[lfq]\naveragine_projection = true\n' > $QRES/align.toml
koth_align $QDATA/koth_runs -o $QDATA/align_avg -c $QRES/align.toml

# 4b. target-only DB: 48 UPS1 + E. coli + human (the known-absent probe)
awk '/^>/{p=(!/^>DECOY_/)} p' \
  $QDATA/sprot_ecoli_human_with_ups_replaced_shuffled.fasta \
  > $QRES/ups_ecoli_human.fasta

# 4c. --matrix-min-runs 6 is REQUIRED, not a tuning choice: without the floor the
#     mass-anchor background rises and the search returns nothing (Section 3.9).
$ROOT/target/release/uno \
  --matrix $QDATA/align_avg/intensity_matrix.tsv \
  --matrix-min-runs 6 --fasta $QRES/ups_ecoli_human.fasta \
  --rt-model $RT --profile-reassign \
  --out-abundance $QRES/ab.tsv --out-lfq $QRES/lfq.tsv \
  --out-proteins $QRES/proteins.tsv
```

```
python3 $ROOT/benchmark/quant/dilution_track.py $QRES/ab.tsv
python3 $ROOT/benchmark/quant/peptide_fc_de.py  $QRES/lfq.tsv 50 0_1
```

The only uno configuration used in the identification benchmark is the digest alignment to ms1searchpy's defaults.

```
# benchmark/uno.toml, values copied verbatim, comments condensed. This is the only
# uno configuration used for every identification benchmark in this manuscript. It
# aligns uno's digest to ms1searchpy's defaults so the comparison isolates the
# engines. ms1searchpy defaults: -lmin 7 -lmax 30 -mc 0 -fmods Carbamidomethyl@C.
# Of the three fields, only max_len (50 -> 30) changes a uno default;
# min_len and missed_cleavages already match it.

[digest]
min_len = 7
max_len = 30
missed_cleavages = 0
```

Every other uno setting took its default value. The defaults, read from the configuration structs rather than transcribed, are:

```
# Every uno setting that has a default, with that default, read from
# DigestConfig::default(), MatchingConfig::default() and ScoreConfig::default()
# rather than transcribed from example_config.toml. The identification benchmark
# overrides only the three [digest] fields listed in the configuration above, two
# of which differ from the default shown here (max_len 50 -> 30; missed_cleavages
# is already 0).

[digest]
# Fully-tryptic with no variable modifications, following the DirectMS1 convention.
missed_cleavages = 0
min_len = 7
max_len = 50
cleave_at = "KR"
restrict = "P"
c_terminal = true
peptide_min_mass = 500.0
peptide_max_mass = 5000.0
max_variable_mods = 2
decoy_tag = "rev_"
generate_decoys = true
static_mods = { C = 57.02146 }
variable_mods = { }
# Interior-residue shuffle against a per-protein residue pool, with a mass shift so
# that a decoy is not a mass twin of its target.
decoy_mass_shift = 11.0
decoy_shuffle = true
decoy_method = "shuffle"
decoy_order = 2

[matching]
ppm_tolerance = 10.0
isotope_offsets = [-1, 0, 1]
min_charge = 1
```

```
[scoring]
# Candidate peptides kept per feature, targets and decoys each, so up to 2*top_k.
top_k = 50

# Mass-anchor stage and the combined mass+RT deviation discriminant.
mass_anchor = true
mass_anchor_ppm = 10.0
discriminant = "deviation"
calibration_fdr = 0.01
bootstrap_rounds = 2
seed_min_runs_frac = 0.5
rt_calibrated = false

# Learned per-PFM rescorer (cross-validated gradient-boosted trees, pure-Rust
# sequoia-boost backend) followed by a cumulative-inclusion protein ensemble.
# Enabled by default and active in every search reported here.
ml_rescore = true
n_folds = 3
iterations = 100
max_depth = 6
shrinkage = 0.1
max_train_rows = 200000
```

S2: Per-dataset and per-replicate results

Table S3 gives every exact value behind Figure 2, including the three ms1searchpy replicates individually and the resource measurements summarized in the main text.

Dataset	Features	Engine	1% FDR	5% FDR	user-CPU (s)	wall (s)	peak RSS (MiB)
09746	59,645	uno	862	1209	350.8	33.0	3414
		ms1searchpy rep 1	837	1126	1457.4	205.8	7096
		ms1searchpy rep 2	853	1122	1405.2	195.2	7075
		ms1searchpy rep 3	823	1042	1411.0	195.2	6976
09766	80,033	uno	875	1100	361.5	33.1	3415
		ms1searchpy rep 1	913	1062	1671.7	233.4	7659
		ms1searchpy rep 2	804	977	1651.0	226.8	7548
		ms1searchpy rep 3	789	989	1675.7	232.1	7603
09734	71,581	uno	1035	1245	353.2	31.9	3317
		ms1searchpy rep 1	770	1073	1436.9	203.7	7090

Dataset	Features	Engine	1% FDR	5% FDR	user-CPU (s)	wall (s)	peak RSS (MiB)
		ms1searchpy rep 2	880	1024	1413.9	195.4	7184
		ms1searchpy rep 3	951	1181	1434.0	203.8	7230
09778	86,778	uno	1222	1653	416.8	40.2	3980
		ms1searchpy rep 1	1173	1660	1647.3	240.1	7610
		ms1searchpy rep 2	1220	1510	1654.0	235.2	7732
		ms1searchpy rep 3	1147	1474	1668.2	239.1	7670
09950	87,019	uno	1440	1740	375.8	37.0	3531
		ms1searchpy rep 1	1169	1645	1678.5	245.3	7532
		ms1searchpy rep 2	1469	1751	1640.1	234.8	7527
		ms1searchpy rep 3	1279	1740	1676.5	247.4	7650
LUM2_05433_1009	100,949	uno	2072	2916	422.7	44.6	4200
		ms1searchpy rep 1	2081	2629	1557.9	243.4	7905
		ms1searchpy rep 2	1481	2286	1537.8	234.4	7877
		ms1searchpy rep 3	1483	2319	1536.4	232.9	7584

Table S3: Per-dataset and per-replicate identification depth, CPU time, wall-clock time and peak resident memory. Depth is the number of human proteins accepted at the stated two-proteome entrapment FDR, not a count from either engine's own reported q. Memory is peak resident set size in MiB, as reported by `/usr/bin/time -v`.

S3: Entrapment calibration

Table S4 holds the per-dataset counts behind the calibration statement in Section 3.3. For each engine it gives the human and yeast proteins on its own reported 1% list, the resulting scale-corrected empirical false-discovery proportion, and the entrapment depth at 1% used everywhere else in this paper. The last two columns differ because the entrapment depth is the deepest cutoff whose entrapment q meets 1%, which need not coincide with the engine's own 1% list. On 09734 the gap is 830 against 1035 proteins.

These per-dataset proportions are quantized and should be read as such. The correction factor is 20,647 divided by 6,066, or 3.40. One accepted yeast protein is therefore worth roughly 0.34 percentage points of estimated FDP at 1000 accepted human proteins, and the accepted sets

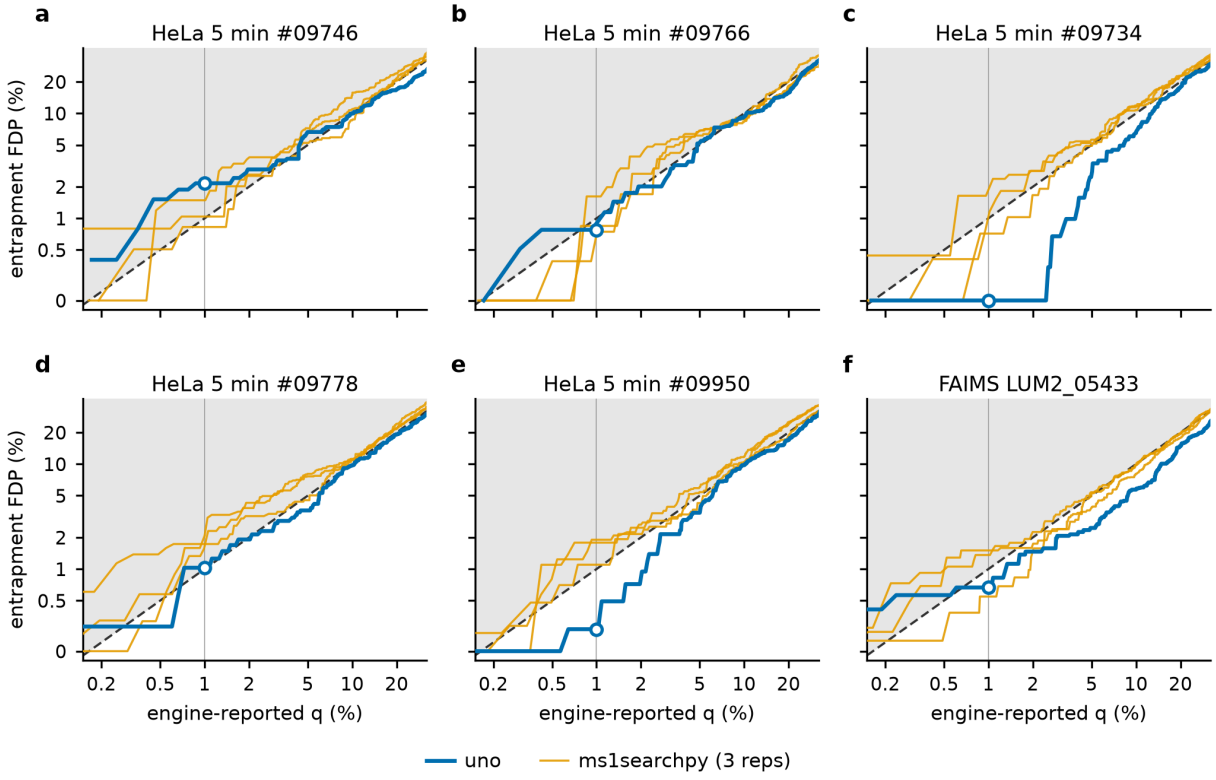


Figure S1: Each engine's own reported q against the entrapment false-discovery proportion of the list it reports at that q, one panel per dataset, for uno and for each ms1searchpy replicate. This is the per-dataset version of Figure 4(a), and the vertical line marks a reported q of 1%, not the entrapment operating point. Individual panels are quantized by the small number of accepted yeast proteins behind each curve.

in Table S4 contain between 0 and 8 yeast proteins. A single yeast hit therefore moves an individual panel of Figure S1 by about as much as the entire difference between the two engines' medians. The pooled median over datasets is the trustworthy statistic, not any one panel.

Dataset	Engine	Human	Yeast	Entrapment FDP (%)	Entrapment depth, 1%
09746	uno	913	6	2.16	862
	ms1searchpy rep 1	884	4	1.49	837
	ms1searchpy rep 2	911	3	1.04	853
	ms1searchpy rep 3	723	2	0.83	823
09766	uno	859	2	0.78	875
	ms1searchpy rep 1	870	1	0.39	913
	ms1searchpy rep 2	786	2	0.85	804

Dataset	Engine	Human	Yeast	Entrapment FDP (%)	Entrapment depth, 1%
	ms1searchpy rep 3	831	4	1.62	789
09734	uno	830	0	0.00	1035
	ms1searchpy rep 1	828	4	1.64	770
	ms1searchpy rep 2	838	2	0.77	880
	ms1searchpy rep 3	929	2	0.72	951
09778	uno	1283	4	1.03	1222
	ms1searchpy rep 1	1337	7	1.73	1173
	ms1searchpy rep 2	1305	6	1.56	1220
	ms1searchpy rep 3	1293	8	2.07	1147
09950	uno	1262	1	0.27	1440
	ms1searchpy rep 1	1271	7	1.78	1169
	ms1searchpy rep 2	1508	5	1.10	1469
	ms1searchpy rep 3	1397	8	1.90	1279
LUM2_05433_VG	uno	1959	4	0.67	2072
	ms1searchpy rep 1	1846	3	0.55	2081
	ms1searchpy rep 2	1728	7	1.36	1481
	ms1searchpy rep 3	1762	8	1.50	1483

Table S4: Each engine's own reported 1% protein list, scored against the entrapment probe.

S4: FAIMS benchmark

The FAIMS comparison of Section 3.1 uses the three replicate 200 ng HeLa Lumos-with-FAIMS runs of PXD023977 and the deposit's own human plus *E. coli* database, with the premade decoy entries stripped so that each engine generates its own. *E. coli* is the absent entrapment species there, so this comparison is independent of the yeast probe used elsewhere.

Engine	1% FDR	5% FDR	Per-replicate, 1%
uno	1591	2060	1642 / 1460 / 1672

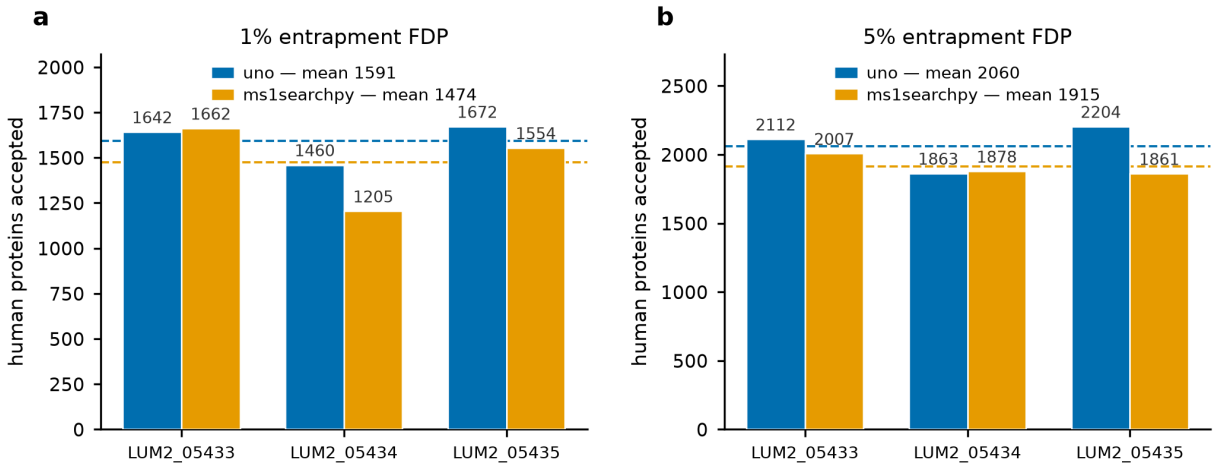


Figure S2: FAIMS benchmark on PXD023977, three replicate runs, *E. coli* entrapment. uno against ms1searchpy at 1% and 5% FDR. uno is deeper on average at both thresholds and on four of the six replicate-by-threshold comparisons, and ms1searchpy's replicate spread exceeds the difference between the engines.

Engine	1% FDR	5% FDR	Per-replicate, 1%
ms1searchpy	1474	1915	1662 / 1205 / 1554

Table S5: FAIMS benchmark, mean over the three replicate runs, with the per-replicate 1% counts in run order LUM2_05433 / 05434 / 05435.

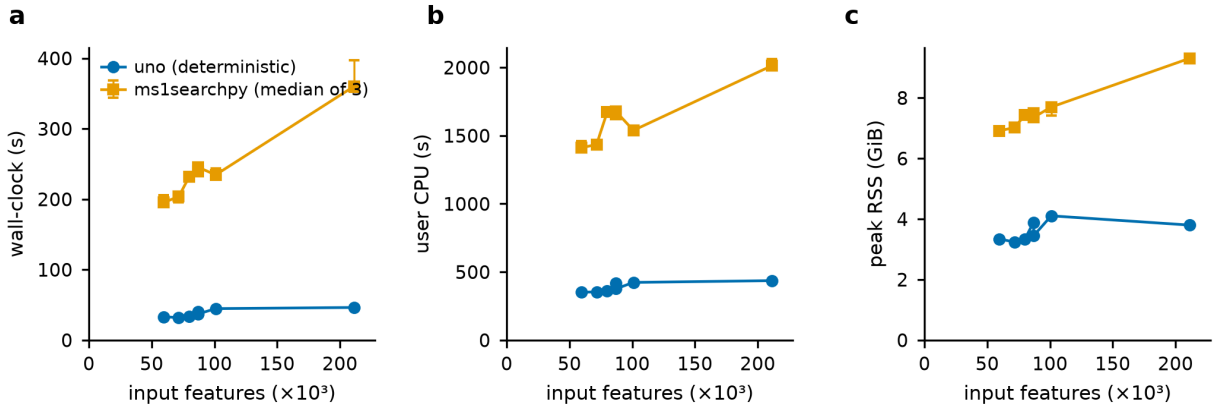


Figure S3: Wall-clock time, user CPU time and peak resident memory against input feature count. uno is plotted as its single deterministic value; ms1searchpy as the median of its three replicates with a bar spanning their minimum and maximum. Seven points are shown per panel, not six: run10 is included, which is the tuning set rather than one of the six benchmark datasets. Both engines grow with input size over the range benchmarked. The separation between them is not constant: the wall-clock ratio spans 5.3 to 7.8 across these points, and uno's peak memory is not monotone in feature count.

S5: Resource scaling

S6: The per-feature quality columns are not a depth lever

The feature finder emits per-feature quality columns (`cosine_score`, `isotope_score`, `combined_score`, `ppm_error`, `neutron_offset`) that a machine-learning rescoring stage can read. Both engines do read them. Neither depends on them, because they carry almost no target/decoy information: the columns describe the feature, and both engines' decoys match real, well-formed features by mass coincidence.

On run10, separating confident-target matches from decoy matches, the columns give an area under the ROC curve of 0.500 to 0.526 in uno and a comparable range in ms1searchpy (Table S6). For uno the value stays at 0.50 even in the full unreduced 41-million-match pool, so candidate reduction is not stripping a signal. There is none at any pool size. Stripping all of the columns from the input changes uno's depth on run10 from 1309 to 1306 at 1% FDR, and from 1887 to 1772 at 5%. The ms1searchpy median falls from 1155 to 1062 at 1%, a drop within its own replicate spread (Table S7, Figure S4). What does separate target from decoy is retention time, with a distance from chance of 0.136 for uno against 0.080 for ms1searchpy, and the full match classifier is further from chance again for both engines (Table S6).

Per-match feature	uno	ms1searchpy
`cosine_score`	0.521	0.514
`isotope_score`	0.521	0.514
`combined_score`	0.526	0.517
`ppm_error`	0.503	0.502
`neutron_offset`	0.500	0.500
retention time ($\lvert\text{AUC} - 0.5\rvert$)	0.136	0.080

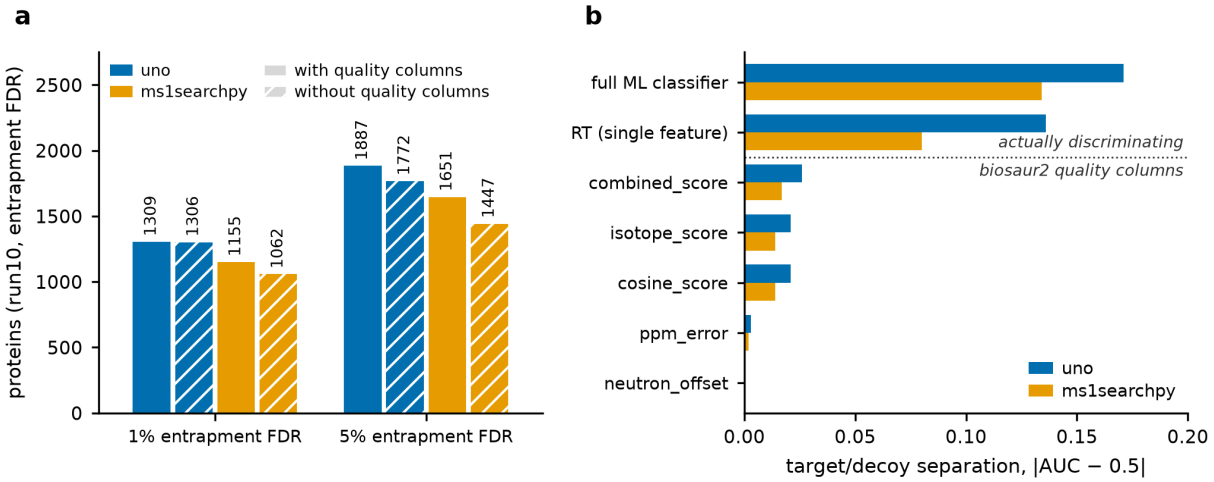


Figure S4: The per-feature quality columns as a depth lever, on the run10 dataset. **(a)** Identification depth with and without the columns, for both engines. **(b)** Per-column separation of confident-target from decoy matches, plotted as the absolute deviation of the ROC area from chance, so that **zero** is no separation and larger is more. Two rows are not quality columns: retention time, and the full match classifier, both included for scale.

Per-match feature	uno	ms1searchpy
full match classifier ($\lvert AUC - 0.5 \rvert$)	0.171	0.134

Table S6: Per-feature target/decoy separation, run10, as recorded from a one-off diagnostic run whose scripts were not retained. The last two rows give the distance from chance for retention time and for the full match classifier, in the form the source records them, rather than a raw AUC.

Engine and input	1% FDR	5% FDR	ms1searchpy replicate range, 1%
uno, quality columns present	1309	1887	n/a (deterministic)
uno, quality columns stripped	1306	1772	n/a (deterministic)
ms1searchpy median, columns present	1155	1651	1079–1220
ms1searchpy median, columns stripped	1062	1447	1023–1134

Table S7: Depth with and without the per-feature quality columns, run10, human plus yeast entrapment. Every row except the column-stripped uno row was recomputed from protein tables still on disk. That row survives only in the benchmark record, because the run behind it was not retained.

S7: Quantitation detail

uno quantifies from koth's averagine-projected intensities (Experimental Section, Quantitation), which project each cell onto its mass-derived isotope pattern and reject the co-eluting off-pattern interference a raw box-sum would absorb. It is the more robust of the two estimators for this task.

The differential contrast behind Section 3.5 is the extreme 50 against 0.1 fmol pair. DirectMS1Quant[16] reports 42 changed UPS proteins there, read from its Figure 2a and accepted at a 5% quantitation FDR against a database without a human entrapment probe. Our own test is stricter on both counts, and the comparison should be read with that in mind. A peptide-fold-change test on uno's matrix recovers 22 of the UPS1 proteins, about half as many. It also accepts 23 *E. coli* and 60 human proteins, and since human is the known-absent probe those are false positives outright, so precision on this contrast is 21%, against a reference output that contains no false positives at all (Figure S5 c). Both numbers are sensitive to how missing cells at the dilute end are imputed, so the contrast is indicative rather than a performance figure. The measurement count is not what limits the test. uno carries slightly more peptide measurements per protein than the reference does, and the shortfall is in the quantitative values feeding the test, whose origin is detection rather than the estimator.

Tracing UPS1 precursors that are confident at 50 fmol down to 0.1 fmol gives two divergent answers whose distance is itself the result. Of the precursors whose aligned-matrix cell is filled at 50 fmol, 82% still have a filled cell at 0.1 fmol, because the aligner transfers a match into runs where the peptide was never independently found. Of the precursors independently detected at 50 fmol, scored at koth's own per-cell q of 0.01 or below, essentially none survive to the dilute end (Figure S5 b). The two percentages have different denominators and are not two views of one population. Transfer of this kind can be brought under explicit FDR control, as IonQuant

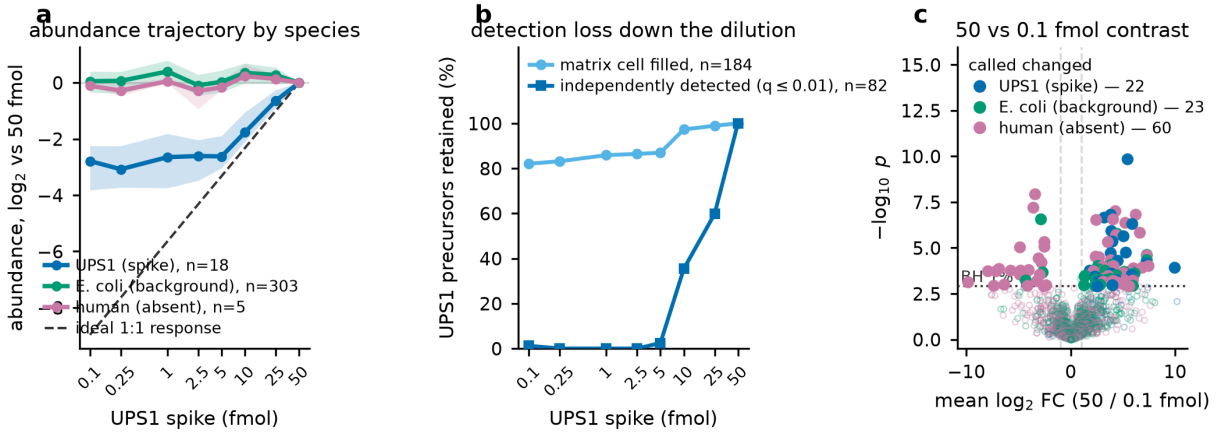


Figure S5: Quantitation detail on PXD033350. **(a)** per-species abundance trajectories across the eight spike levels. The trajectories separate the spike from the constant and the absent species. **(b)** detection-loss traces that follow spike-in precursors confident at 50 fmol down to 0.1 fmol. The two traces are the aligned matrix cell staying filled and the precursor being found independently at a per-cell q of 0.01 or below, and the gap between them is match-between-runs over-fill. **(c)** the volcano of the 50 against 0.1 fmol contrast, limited by detection at the dilute end and reported as indicative rather than as a precision figure.

does for match-between-runs[21], which koth's aligner does not currently do. A fold change built on transferred fills rather than genuine detections is fragile, which is why the dilution trend, robust to over-fill, is the primary quantitative result of Section 3.5.

For a substantial minority of the precursors lost, the signal is physically present but sub-threshold. Dilute peptides elute in only 1 to 2 scans, and koth's hill-builder (`min_scans = 2`) discards their traces. Lowering the threshold to `min_scans = 1` does not help. It multiplies the feature count by 6.4 with noise, and an end-to-end test on the six extreme-contrast runs confirms a net loss. At the looser minimum-runs floor used for that test, the random-match background rate p rises from 0.15 to 0.51, so half of all features match something by chance and uno accepts no proteins at all at 1% FDR. At the stricter floor used for the reported results the search still returns proteins, but precision falls and the entrapment FDP rises, so the conclusion is the same in both arms. Untargeted detection cannot separate a one-scan peptide from one-scan noise.

Species	Role in the design	n	Median ρ	Fraction $\rho > 0.5$
UPS1	spike, 0.1–50 fmol	18	+0.71	89%
E. coli	constant background	303	+0.02	4%
Homo sapiens	known absent, entrapment probe	5	+0.25	0%

Table S8: Dilution tracking by species on PXD033350. ρ is the Spearman correlation of protein abundance against log spike amount, and n is the number of proteins quantified in at least 8 of the 24 runs.

S8: Run-to-run reproducibility detail

Every source of run-to-run nondeterminism found in uno's development traced to Rust's process-randomized hash-map iteration order (Section 3.2). The affected paths were the RANSAC retention-time calibration seed, the retention-time width estimate, the mass-anchor

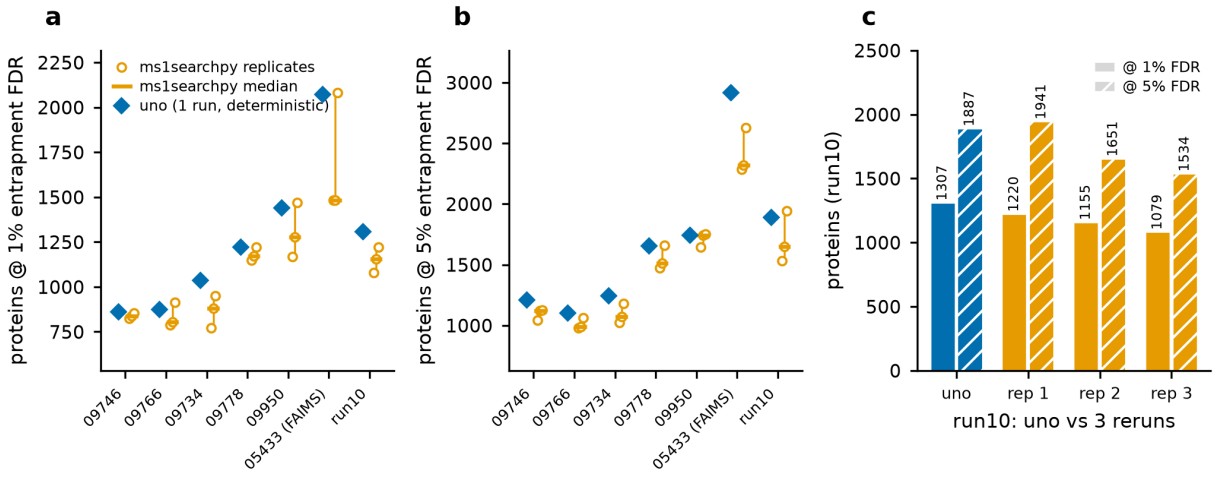


Figure S6: Run-to-run reproducibility on fixed input. **(a)** and **(b)** plot the three ms1searchpy replicates against uno's single deterministic value at 1% and at 5% entrapment FDR. **(c)** is a close-up of run10, the tuning set. Repeating the identical search moves ms1searchpy's answer by 30 to 600 proteins at 1% FDR depending on the dataset, while uno's output is byte-identical between runs. The scale-free form of the spread, the replicate range as a percentage of each dataset's own median, is panel (c) of Figure 2 in the main text.

recalibration sample, the peptide list published to the quantitation rollup, the feature-competition and profile-reassignment outputs, and the protein-group table. Each was made order-independent by sorting on value before reducing, or on accession where a tie genuinely had to be broken. Figure S6 plots the resulting contrast dataset by dataset.

S9: Development-configuration experiments: retention-time model and protein rescorer

The experiments in this section were made on the earlier development configuration (run QEHF1_09744_JB of PXD015669, koth-called features, the distilled MLP retention-time model; Methods, Benchmark data). Their values are not comparable to the six-dataset benchmark.

The distilled student model encodes a peptide by a 99-dimensional order-aware feature set: amino-acid composition (20), N- and C-terminal residue identities (first three and last three, 6), position-binned composition (three sequence bins $\times$ 20 = 60), and 13 scalar descriptors, namely length, ln mass, Kyte–Doolittle[22], Krokhin-SSRCalc[23] and Eisenberg[24] hydrophobicity sums, the mean, maximum and minimum Kyte–Doolittle value, an N-terminus-weighted SSRCalc term, proline, aromatic and basic counts, and a hydropathy moment. A small multilayer perceptron over these features, trained on DeepLC predictions over a large, diverse tryptic digest, is the distilled student.

A naive composition-feature model trailed at $R^2 = 0.972$ against DeepLC predictions. Its encoding followed the composition-only approach of Klammer et al.[25]: amino-acid counts, counts within fixed N- and C-terminal windows, peptide length and mass. It gave 141 proteins against the DeepLC library's 176, and fewer of them overlapped the reference search (Table S9). Replacing the encoding with the 99-d order-aware feature set raised the distilled model to $R^2 = 0.980$, matching the LightGBM ceiling, and restored identification depth to DeepLC parity

(Table S9), in-process and in pure Rust. The feature encoding, not the boosting backend, was the lever. A still-richer 215-d set gave only marginal improvement ($R^2 = 0.9824$), indicating that the 99-d set is near the practical ceiling for hand-crafted features.

RT model	R^2 vs DeepLC	Proteins / ref-overlap	RT-stage cost	Dependency
Composition-only	0.972	141 / 119	in-process	none (Rust)
Order-aware (99-d, this work)	0.980	~180 / ~152	in-process	none (Rust)
DeepLC library	(teacher)	176 / 152	~130 s preproc.	Python

Table S9: RT model against identification depth at 1% FDR, measured on the earlier development configuration (run QEHF1_09744_JB of PXD015669, koth-called features). Reference overlap is the number of accepted proteins also found by the DeepLC-driven reference search.

A reverse-sequence benchmark quantifies how much sequence-order information a model captures. A peptide and its reverse have identical amino-acid composition but different true retention time. That sequence order carries retention information beyond composition is long established[26], and a retention-coefficient model is by construction unable to represent it, since it predicts one value for any permutation of a given composition[27]. An order-aware predictor must assign a peptide and its reverse different RTs, while a composition-only model is forced to predict the same value for both. This forward minus reverse RT difference is the signal that lets a search engine distinguish a real peptide from a reversed decoy, and the benchmark reports how much of it a given model actually captures.

The benchmark holds 46,304 held-out tryptic peptides and their reverses, which are non-tryptic and out of distribution. DeepLC RT was predicted for both, and each model was measured on how well it reproduced the forward minus reverse RT shift. A composition-only model captured 0% of this signal, by construction. uno's order-aware model captured 72% (Pearson $r = 0.68$) while predicting reverse-sequence RT nearly as well as forward ($R^2_{\text{rev}} = 0.96$, $R^2_{\text{fwd}} = 0.98$, Table S10). The residual 28% is the genuine gap between hand-crafted features and DeepLC's sequence convolution.

Model	R^2_{fwd}	R^2_{rev}	order-Δ corr.	order magnitude
Composition-only	0.96	0.95	0.00	0%
uno order-aware (this work)	0.98	0.96	0.68	72%

Table S10: Reverse-sequence RT benchmark, 46,304 held-out tryptic peptides and their reverses, on the earlier development configuration.

The hardened variant of the LDA protein rescorer of Section 3.4 applied high-confidence iterated positives ($q \leq 0.01$, refreshed), target/decoy pairs co-folded, and stronger covariance shrinkage. It removed the inflated apparent count on the human-only search (242 to 223) but did not restore FDR control, which stayed at 11.4% empirical under the entrapment probe (Table S11). These safeguards follow established semi-supervised rescoring practice[28,29].

Protein scorer	Real accepted	Yeast (false)	Empirical FDR
Binomial (default)	201	1	1.7%

Protein scorer	Real accepted	Yeast (false)	Empirical FDR
LDA rescorer (naive)	208	7	11.5%
LDA rescorer (hardened CV)	209	7	11.4%

Table S11: Two-proteome entrapment of three protein scorers, measured on the earlier development configuration. Empirical FDR is the database-size-corrected entrapment estimate at a reported 1%. The binomial is the shipped default; both LDA variants are disabled.